PROJECT REPORT

Reliable Market-Data Distribution for a Networked Electronic Exchange

GROUP NO. 11

Course: Computer Networks (CSC-303)
Department : Computer Science and Engineering
Project Guide: Dr. Sandeep Kumar Garg

STUDENT DETAILS AND INDIVIDUAL CONTRIBUTIONS

S. No.	Student name	Enrollment	Email ID	Contributions
1	Anurag Kishor Patil	24114015	anurag_kp@cs.iitr.ac.in	Designed overall system architecture, exchange structure, order-book design and integration of the main components.
2	Prabhat Chandra Tiwari	24114066	prabhat_ct@cs.iitr.ac.in	Developed the order-entry mechanism, TCP communication and integration of client orders with the exchange and matching engine..
3	Vaibhav Kumar	24114103	vaibhav_k@cs.iitr.ac.in	Market-data publishing and subscription, including the dual-feed mechanism and UDP multicast communication.
4	Samarth Maheshwari	24114084	samarth_m@cs.iitr.ac.in	Worked on sequence tracking, duplicate detection, packet-loss identification and recovery of missing market-data events.
5	Vishal Kumar Shaw	24114105	vishal_ks@cs.iitr.ac.in	Designed Snapshot-based recovery, SHA-256 integrity verification, recovery history and recovery scheduling.
6	Rishabh Gupta	24114077	rishabh_g@cs.iitr.ac.in	Created the graphical dashboard, performance metrics, fault-injection experiments, testing and result analysis.

Abstract

Electronic Market Exchanges depend on two distinct but interrelated internet communication functions: the receipt of trading orders and the distribution of market data to the subscribers. Order entry has to be able to reliably pass a client's order to the market exchange and get the order book updated and market data distribution has to be able to provide a timely and consistent view of the market to a large number of subscribers. UDP multicast is ideally suited to a one-to-many communication as it doesn't require a separate stream to be maintained for each subscriber, but the limitations of UDP is that it does not guarantee delivery, ordering or duplicate suppression. A subscriber therefore could potentially end up with an incomplete local view of the order book.

This project is a networked electronic exchange prototype and a reliability layer for market data distribution. It maintains One limit order book per instrument(Company) and accepts orders for buy and sell reliable TCP. Order book updates are converted into sequence-numbered market data events which are then disseminated via UDP multicast. The reliability layer consists of two multicast feeds, sequence tracking, duplicate suppression, bounded event history, TCP replay, snapshot fallback, SHA-256 checkpoint verification, and recovery scheduling. The evaluated metrics such as telemetry is shown in CSV export, automated experiments on a PyQt6/pyqtgraph dashboard interface.

The implementation contains both an in-process deterministic reliability demo and a separate multi-process live path using actual TCP and UDP sockets. The test results are 51 passing unit tests, live end to end scenarios, 30 consecutive lossy run passes, tests with up to 32 subscribers and measured scheduler and dual feed results. These are the repository test results as reported, not as independent production exchange benchmarks.

1. Introduction

An electronic exchange is a computer system that takes buy and sell orders, decides which orders can trade against each other, maintains the current state of sells and buys in the market, and publishes information about these changes. Matching engine of an exchange keeps authoritative state of the market. A market-data subscriber (for example, a broker, trading software, analytics service or research system) takes published information and builds its own local picture of order book.

The problem lies in the fact that order entry and market data flows have different demands. Order entry flow is inherently request-response protocol: a client sends an order and expects either an acknowledgment, rejection or response regarding the execution of this order. Market data is a high volume one-to-many flow where the same event has to reach many subscribers fast. TCP guarantees ordered reliable delivery on a per-connection basis; however, having a separate TCP connection for each market data subscriber is inefficient. UDP Multicast allows efficient one-to-many delivery, yet packets could be lost, duplicated or out of order.

The project solves this problem. It does not try to implement distributed geographically exchange and replicated matching engine based on consensus. Rather, it retains a central authoritative exchange and makes the downstream market-data distribution path resilient to loss and temporary subscriber disconnection.

1.1 Background and domain concepts

Limit order book: a per-instrument order book consisting of resting bids (buy orders) and asks (sell orders) sorted by price and time priority. The best bid is the highest buy order, while the best ask is the lowest sell order. A matching engine tries to match incoming orders with opposite-side orders and execute them.

Price-time priority: among orders with the same price, the one which was resting earlier has higher priority. The repository keeps bid levels sorted in decreasing price order, while ask levels are sorted in increasing price order. A limit order tries to be matched against orders in the other side with opposite prices first and, if not fully executed, remains in the book. Market orders are kept in such a way that they cross opposite-side levels.

Market data event: a message about a certain change on the book or trades. Every market data event has a sequence number increasing monotonically. It allows the subscriber to determine if it has missed an event, got an outdated copy, or got an event later than some other.

UDP multicast: a single datagram published to a multicast group and port will be received by members of the same group. The network delivery is unreliable, i.e., there is no guarantee that all datagrams will be delivered. That is why sequence tracking and a mechanism to repair the sequence are needed.

Replay and snapshot: replay delivers stored original events to fill a known gap in the sequence. Snapshot – is a checkpoint of book state at a certain sequence point. Replay is efficient in cases of short sequence gaps left in history. A snapshot comes handy in case when the missed events are not available in bounded history anymore.

Integrity Digest: a fixed-size digest of canonical data will be produced via SHA-256. In this project, a snapshot hash consists of canonical checkpoint data and previous checkpoint hash. It makes an integrity chain but it is neither a persistence measure nor a publisher authentication method, nor it prevents any tampering with data.

1.2 Motivation

The subscriber could get sequence 100 followed by sequence 102. In the absence of a repair function, it will not know whether sequence 101 is irrelevant or not because the event could have modified the best price, withdrawn liquidity, or made a trade. The actions that will follow could then be based on old and inaccurate market information.

There can be a second multicast feed which will minimize the occurrence of gaps that need to be repaired since even when an event is missed on feed A, it can still appear on feed B. In case the two feeds miss out on an event, then bounded history can enable retransmission via TCP. For cases where the event is old and there is no history that has been retained for it, the snapshot can restore the state.

1.3 Problem statement

Create a prototype electronic exchange where trading customers place their orders using TCP, limit order books are kept by the exchange, and market data consumers receive the sequence number-based updates using dual feed UDP multicast. The system should be able to identify losses, duplicates, and out-of-order arrival of updates; retrieve lost updates using a backup stream or TCP replay mechanism; revert to snapshotting if there is no replayable history; validate the reconstructed state; and provide verifiable proof via tests, measurements, and visualizing dashboards.

1.4 Objectives

The main objectives are:

(i) implement a central order-book and matching-engine workflow

(ii) separate TCP order entry from UDP market-data dissemination

(iii) provide dual market-data multicast feeds

(iv) detect duplicates and sequence gaps

(v) recover gaps from bounded history over TCP

(vi) use periodic or on-demand snapshots when replay is too old

(vii) verify snapshot chains and final book state with SHA-256 digests

(viii) compare FCFS and Round-Robin recovery scheduling

(ix) inject repeatable faults and collect meaningful metrics

(x) provide a GUI for operation, experiments and result inspection.

1.5 Application domain and potential use

The architecture is applicable for market data gateways, broker feed handling services, exchange technology labs, financial networking courses, and any publish/subscribe infrastructure where consumers build their own state based on events in an ordered sequence. The same basic architecture can be used for telemetry delivery, industrial monitoring, multiplayer games, and replicated read models, assuming that the application can make necessary adjustments in terms of consistency and security.

This repository is an educational and experimental prototype. It cannot be construed as a certification exchange, trading venue, or replacement for controls necessary in a real-life financial market.

2. Proposed System

The proposed system uses a centralised exchange with one authoritative order book per instrument. Trading clients connect to the order-entry service using TCP service. The exchange validates and processes incoming sell or buy orders, updates the appropriate order book, and converts book changes into market-data messages. A Market data publisher assigns sequence numbers and places messages in a ring buffer for asynchronous publication.

The live networked path can multicast the same sequence-numbered datagram stream to Feed A and Feed B. Subscribers listen to one or both feeds, suppress duplicates, track sequence continuity, buffer event which are not in order while an earlier sequence is missing, and request the events corresponding to the missing sequence number. A separate TCP recovery endpoint serves retained events or a snapshot depending upon the length of messages requested. The subscriber applies a contiguous stream of events to the BookBuilder and finally performs comparison between the local and exchange digest.

There is also a reliability demonstration tool working deterministically in-process. It generates small dummy messages called heartbeat packets sent periodically between two systems via callback-based feeds and recovery objects without any UDP/TCP sockets. This tool is helpful for testing algorithms repeatedly.

2.1 High-level architecture

Figure below is the high level project architecture depicting the working of the proposed system. It emphasizes fault-injection experiments, dual multicast market-data feeds, subscribers, telemetry, TCP recovery and snapshot recovery.

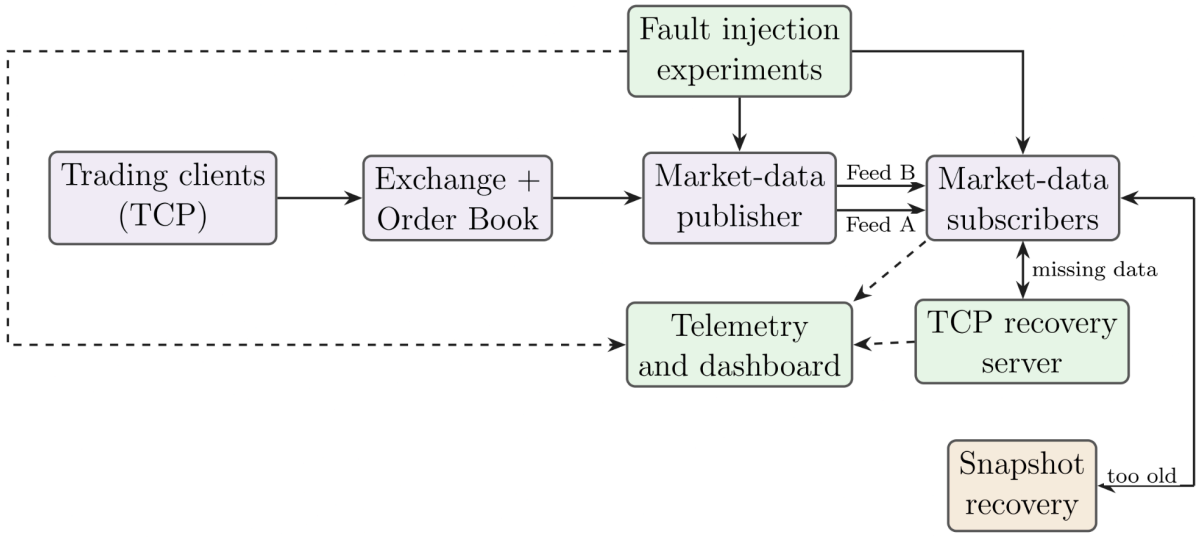


Fig 1. High level architecture design

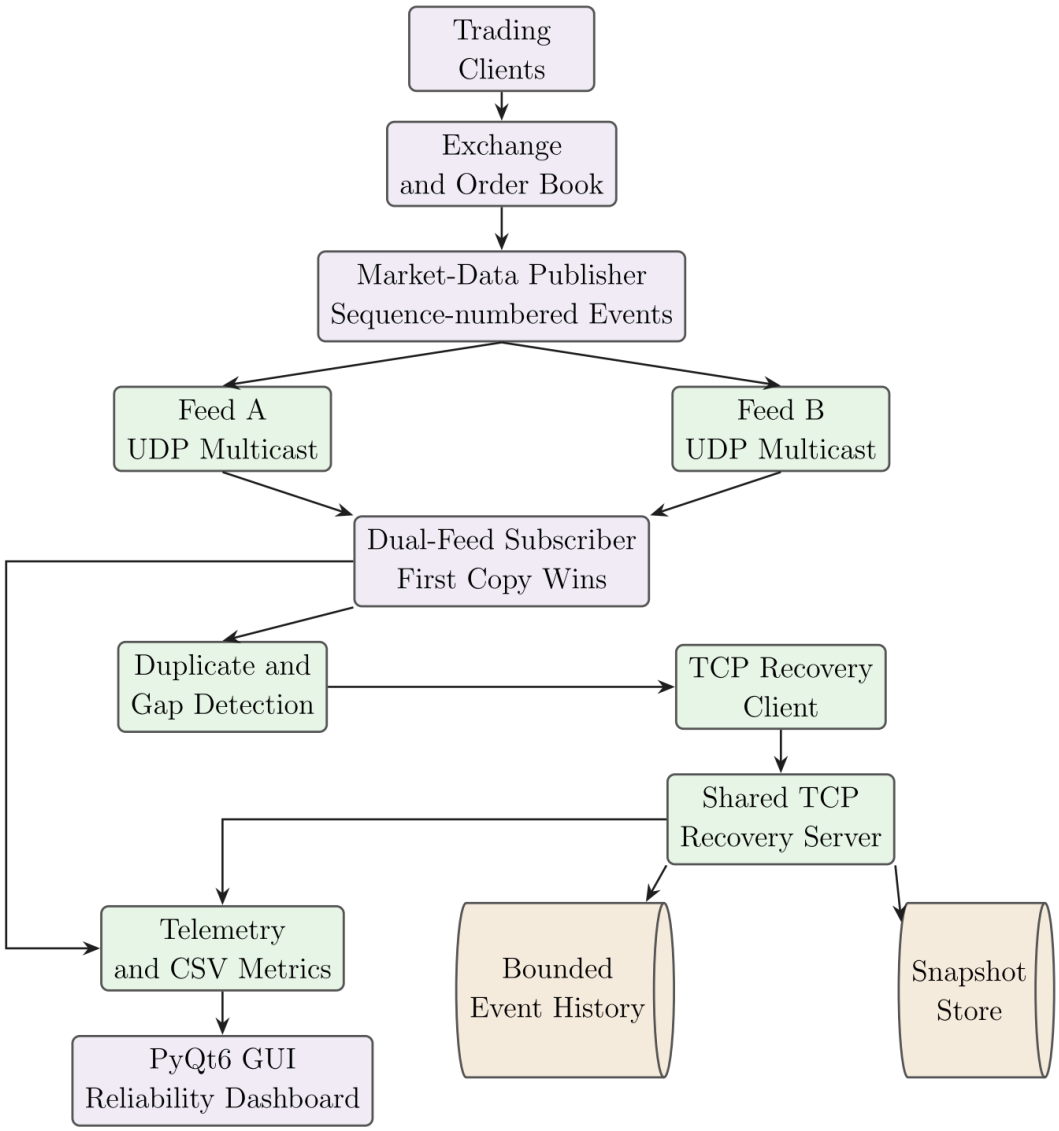


Fig 2. Pipeline architectural view

2.2 Main subsystems

A. Trading clients and order entry. Interactive and automated clients establish TCP connections to the exchange. The order-entry protocol uses framed binary messages. The server reassembles the TCP byte stream into complete frames, validates message sizes, handles multiple clients, and places accepted messages into a bounded mutex-protected queue. The exchange consumes this queue and processes new, cancel and compatible modify messages.

B. Exchange and order book. The exchange owns the authoritative order books for listed companies such as AAPL, GOOGL, AMZN, NFLX and META. SymbolManager maps ticker symbols to security metadata, market-data adapters and order books. The OrderBook maintains

bid and ask price levels, performs matching, supports cancellation and amendment, and exposes market-depth information. Prices are stored as integer cents; for example, 15050 represents USD 150.50.

C. Market-data publisher. Order-book adapters notify the publisher about price-level changes and trades. The publisher builds packed binary messages with a global sequence number. A ring buffer separates the order-book notification path from the background publisher thread. This avoids requiring the matching path to perform all network transmission synchronously.

D. Dual multicast feeds. The live exchange can send the same datagram stream to two configured multicast destinations: Feed A at 239.1.1.1:9999 and Feed B at 239.1.1.2:10000. The feeds are redundant delivery paths; they do not represent two independent matching engines. A subscriber using both feeds accepts the first valid copy of a sequence and treats the later copy as a duplicate.

E. Subscriber and sequence tracking. The reliable subscriber receives multicast datagrams, applies configured receive-side impairments where enabled, records per-feed arrivals and passes events to the dual-feed subscriber. SequenceTracker compares each event with the next expected sequence. Older-than-expected events are duplicates; a higher-than-expected event indicates a gap. Later events are buffered until missing sequences arrive, so the local book is updated in order.

F. TCP recovery service. When a gap is detected, the subscriber waits for a short gap-grace interval to allow the other multicast feed to supply the missing event. If the gap remains, the subscriber requests the missing range from the recovery server. The server consults bounded RecoveryHistory. It can return retained original datagrams, chunk large responses, or report TOO_OLD when the requested sequence is no longer available.

G. Snapshot fallback and verification. If replay is too old, the subscriber requests a snapshot. The exchange can use a retained checkpoint or create an on-demand checkpoint when enabled. Snapshot fields are canonicalized and hashed with SHA-256 and the previous checkpoint hash. The subscriber verifies the hash before applying the checkpoint and resynchronising its expected sequence. At end of run, the subscriber obtains the exchange's latest sequence and book digest, drains remaining gaps and checks whether its local digest matches.

H. Recovery scheduler. The recovery server supports First-Come, First-Served (FCFS) and Round-Robin scheduling across subscriber queues. Configurable worker counts and service-delay settings allow contention experiments. Per-subscriber queue-wait and fairness metrics help determine whether a noisy subscriber can delay quieter subscribers.

I. Fault injection and experiments. The system can model loss, duplication, reordering and delay in the live receive path; the deterministic in-process harness also supports repeatable loss and duplication. Random seeds make selected experiments reproducible. Preset experiment runners sweep redundancy, scheduler contention, history/snapshot settings, snapshot tampering and subscriber scalability.

J. Telemetry and dashboard. Structured events and periodic statistics are collected from exchanges and subscribers. CSV files preserve run-level and latency-level results. A PyQt6/pyqtgraph GUI provides market visualisation, exchange control, manual order testing, LAN setup, saved-results inspection and a six-chart reliability dashboard.

2.3 End-to-end operating sequence

The exchange starts, initializes supported symbols and order books, starts its market-data publisher thread, and starts the recovery server. In client mode it also starts the order-entry TCP server.

Trading clients connect and complete the connection identity handshake. They submit framed order messages over TCP.

The order-entry server validates and queues each complete message. The exchange's client-order loop consumes the queued message, resolves the instrument, and calls the appropriate order-book operation.

The matching engine attempts to execute the incoming order against the best opposite-side prices. Any unfilled limit quantity rests in the book. The order-entry path sends an acknowledgement, rejection, or approximate execution report as applicable.

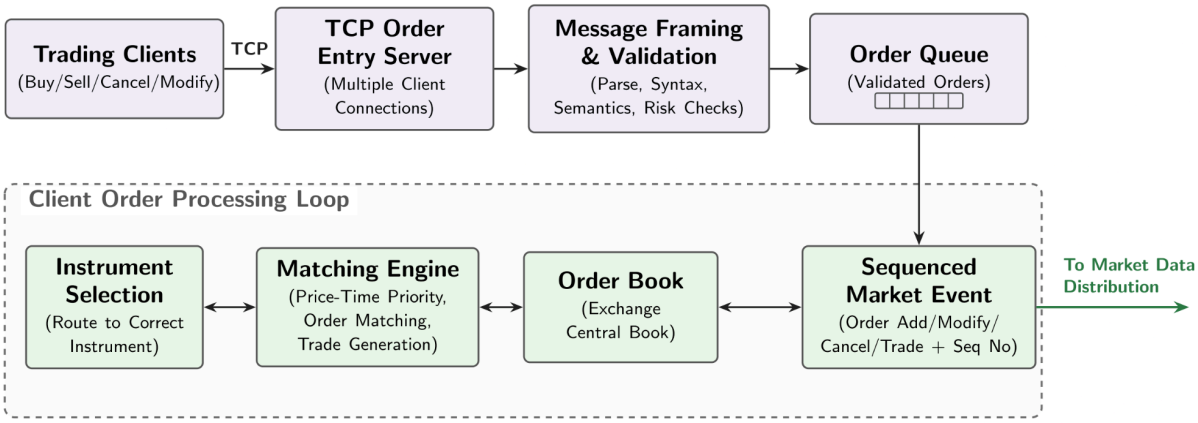


Fig 3. Exchange and Order Processing

Book changes are reported through the instrument's MDAdapter. The MarketDataPublisher creates sequence-numbered messages and enqueues them for the multicast publisher thread.

The publisher sends the stream to Feed A and, when dual-feed mode is enabled, Feed B. Subscribers receive datagrams independently on the two feeds.

The subscriber records arrivals and applies first-copy-wins duplicate suppression. Contiguous events are applied to BookBuilder. Events arriving beyond a gap are buffered and cause the missing range to be identified.

The subscriber gives the second feed a chance to fill the gap. Remaining missing ranges are scheduled for TCP recovery. Recovered events are buffered if necessary and released only when sequence continuity is restored.

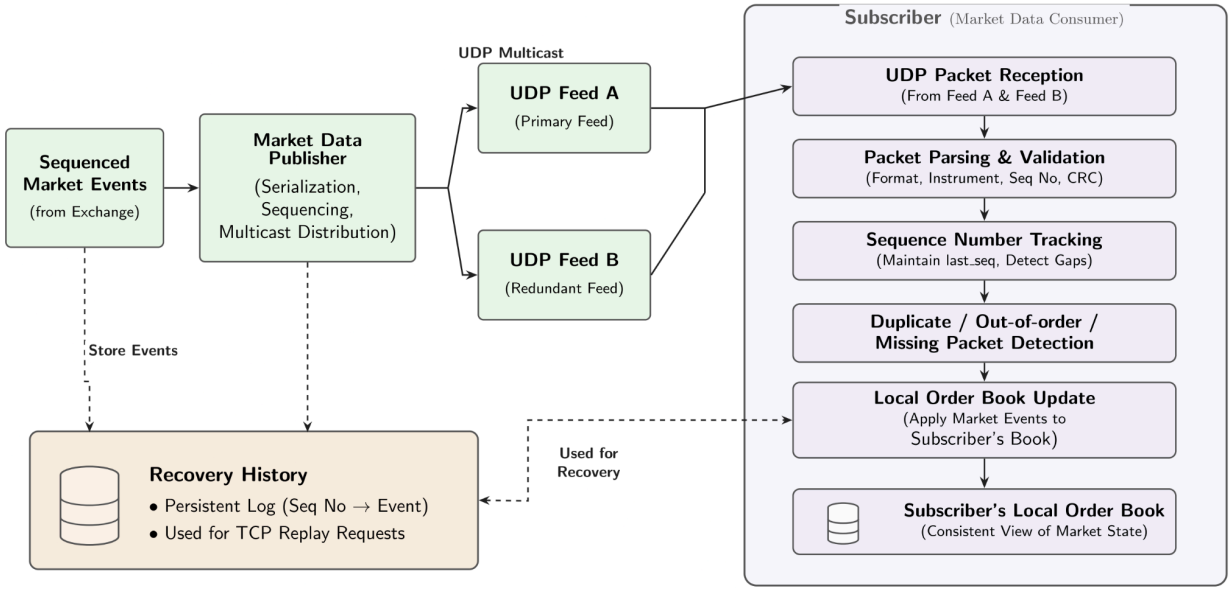


Fig 4. Market Data Distribution

If the requested range has aged out of bounded history, the server responds TOO_OLD and the subscriber requests a snapshot. The subscriber verifies the snapshot hash and applies its book state and checkpoint sequence.

At shutdown, the subscriber stops UDP reception, polls STATUS until the exchange state is stable, informs the tracker of the final sequence, drains the tail gap, verifies accounting and compares local and exchange book digests.

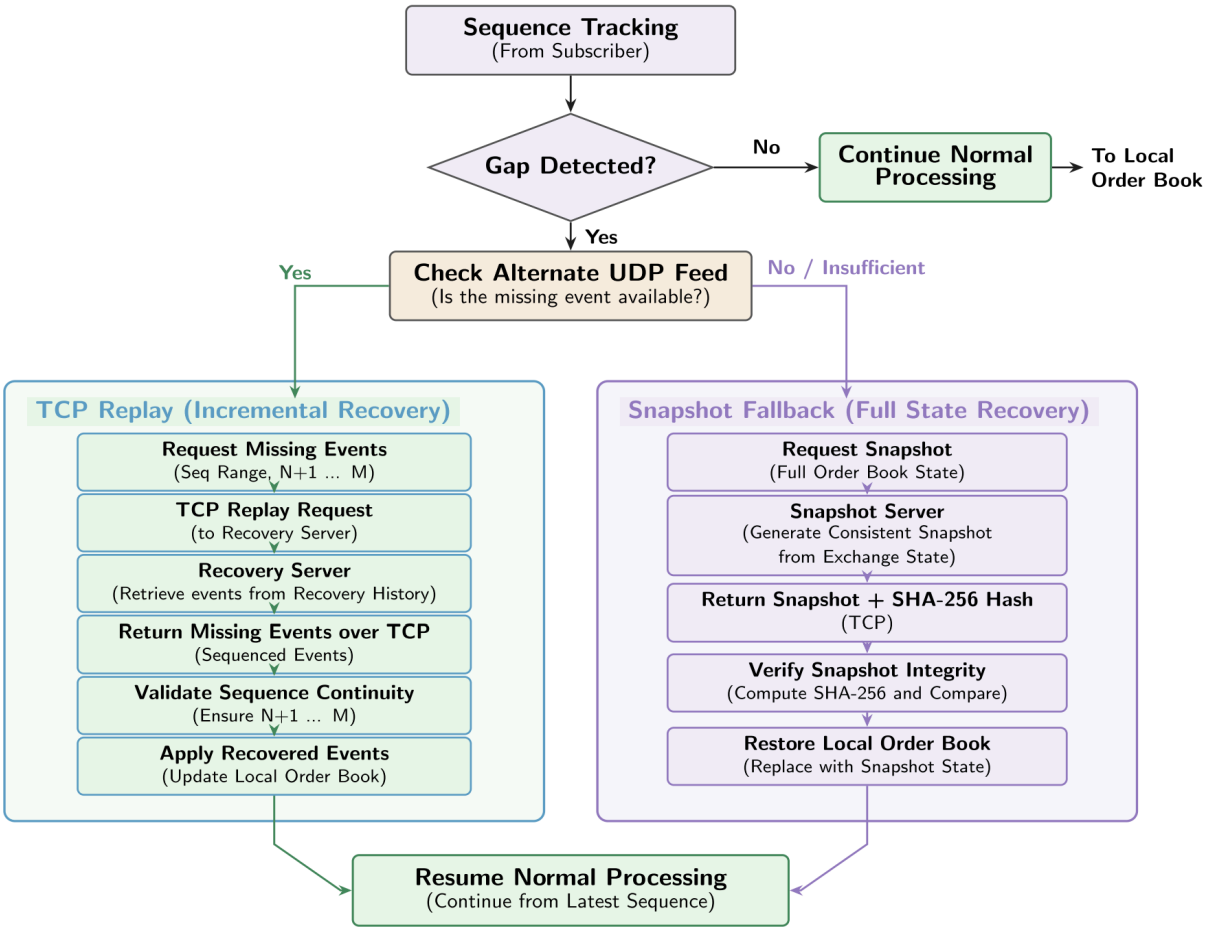


Fig 5. Subscriber Processing and Reliability/Recovering

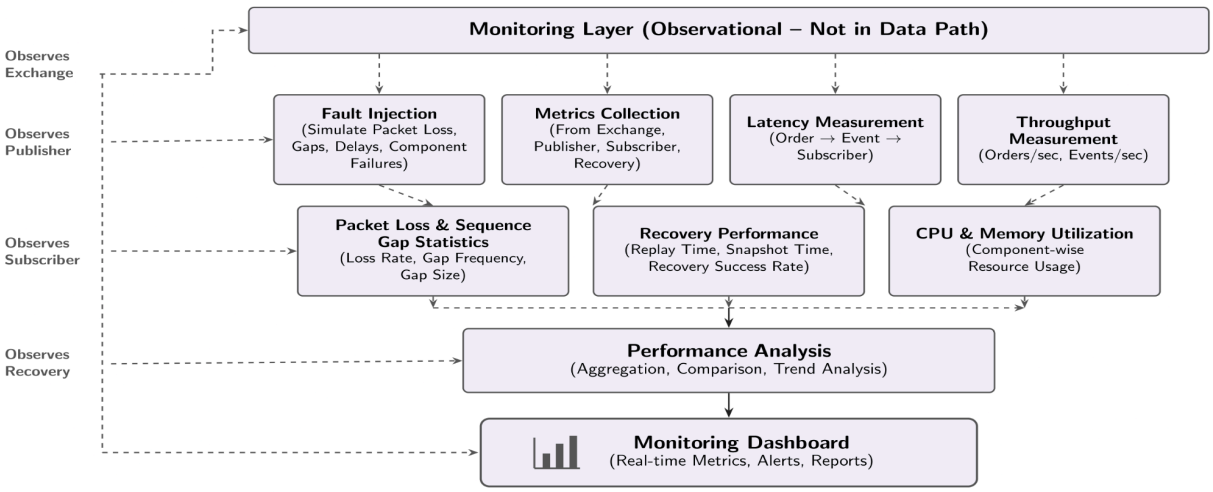


Fig 6. Monitoring and Performance Analysis

2.4 Algorithms and core logic

(i) Sequence tracking and gap detection. Maintain expected sequence E. For each received event with sequence S: if S < E, classify it as a duplicate/old event; if S = E, deliver it and advance E; if S > E, record missing interval [E, S−1], buffer S, and request repair. After a missing event is recovered, advance E and drain any now-contiguous buffered events. The implementation also handles out-of-order recovered slices by buffering them rather than discarding them.

(ii) First-copy-wins dual-feed handling. Each event carries a sequence number and feed identity. The first arriving copy is eligible for delivery. If the same sequence arrives from the other feed later, it is counted as a duplicate and ignored. If one feed misses a sequence but the other delivers it, the second feed avoids a TCP recovery request.

(iii) Bounded replay. Given a missing range [a,b], validate the request and check whether the complete requested range is retained in history. If available, return the original datagrams, using chunked frames for large responses. If the requested data is too old, return TOO_OLD so the subscriber can use snapshot fallback. History is bounded to prevent unbounded memory growth; the oldest entries are evicted as new events arrive.

(iv) Snapshot fallback. On TOO_OLD, request a checkpoint. The server selects a suitable retained checkpoint or builds a current on-demand checkpoint when configured. The checkpoint includes sequence and book-level information, plus its integrity hash and previous-hash link. The subscriber verifies the hash, applies the snapshot, sets its next expected sequence to checkpoint sequence plus one, and then processes later buffered events.

(iv) Recovery fairness. FCFS serves recovery requests in arrival order. Round-Robin selects requests across active subscriber queues so one subscriber with many outstanding ranges is less likely to monopolize service. The experiment measures queue wait by subscriber and a saturated-window Jain fairness index under contention.

(v) Final reconciliation. Stop receiving new UDP packets, obtain a stable exchange STATUS, let L be the exchange's latest sequence, notify the tracker of L, recover any remaining tail gap, and verify that every expected sequence is accounted for. Then compare the subscriber's canonical local book digest with the exchange-provided digest. A successful process exit requires integrity and accounting checks to pass.

2.5 Novelty and innovation

The project's main novelty lies in integrating redundant market-data distribution, multi-level recovery, composite snapshot restoration, integrity verification and fairness-aware scheduling into one working exchange prototype. Its automated experiments provide a framework for evaluating these mechanisms under different network conditions.

Layered Recovery Instead of a Single Repair Mechanism

Rather than depending entirely on retransmission, the system uses a staged recovery strategy. It first benefits from redundancy in the delivery paths, then uses retained event history, and finally restores state through a snapshot when individual missing events are no longer available. This creates a recovery path for both short interruptions and longer periods of data loss.

Consistent Restoration Across Multiple Instruments

The exchange maintains separate order books for multiple instruments while publishing events under a common global sequence. The snapshot mechanism addresses the challenge of restoring these books together at a common sequence checkpoint. This avoids treating each instrument as an unrelated recovery process and supports reconstruction of a consistent multi-instrument state.

Recovery Fairness as a Measurable Design Goal

Recovery is not evaluated only by how quickly the system repairs missing events. The project also examines how recovery work is distributed among subscribers when several request repairs simultaneously. Comparing FCFS and Round-Robin using subscriber-level waiting times and Jain's fairness index makes resource allocation an explicit part of the reliability evaluation.

Reproducible Reliability Evaluation

The project provides a controlled environment for examining how different recovery strategies behave under network impairments and subscriber workloads. Fixed fault seeds, configurable experiments, per-event measurements and structured result exports allow different configurations to be compared using the same evaluation approach. This makes reliability behaviour observable and testable rather than relying only on successful demonstration runs.

Bounded Resource Management with Recovery Continuity

The event history and recovery queues use bounded storage rather than growing indefinitely. When old events are evicted, the snapshot fallback provides another way to restore the subscriber's state. This connects resource management with recovery correctness.

Overall the project brings together layered recovery, multi-instrument state restoration, subscriber fairness, end-to-end correctness checks and reproducible experimentation in one networked exchange prototype. Its contribution is the integrated design and evaluation of these mechanisms, rather than the invention of UDP multicast, TCP, SHA-256 or the scheduling algorithms themselves.

2.6 Requirements and software-engineering process

The project follows a practical software-engineering workflow: requirements and architecture documentation; decomposition into libraries and executables; interface and protocol design; iterative implementation; unit and integration testing; performance and fault experiments; defect tracking and correction; and user-facing documentation and demonstration scripts.

Functional requirements include order submission, matching, cancellation, market-data publication, dual-feed reception, duplicate/gap detection, replay, snapshot fallback, integrity verification, scheduling and metrics. Non-functional requirements include bounded memory, sequence correctness, low recovery delay, multi-subscriber operation, repeatable experiments, portable C++ builds and observable operation.

The repository contains a design document, design errata, implementation checklist, recovery protocol specification, metrics definitions, live-testing instructions, verification ledgers and a viva demonstration guide. The Round 2 verification ledger records findings F0–F15 and corresponding fixes/tests. This traceability is a key part of the project's engineering process: issues such as tail-gap races, out-of-order recovery, missing retries, inaccurate latency measurement and snapshot fallback gaps were identified and addressed through targeted changes and regression tests.

The lifecycle used in the project is: (1) problem identification and scope; (2) requirement analysis; (3) architecture and protocol design; (4) modular implementation; (5) unit testing; (6) multi-process integration and fault testing; (7) metrics and performance evaluation; (8) GUI and documentation; and (9) final verification. It is an iterative development process rather than completed once in a waterfall sequence.

3. Implementation

The implementation is primarily C++23, with Python used for experiment orchestration and the graphical interface. CMake manages the native build. Boost and spdlog are listed as build dependencies; GoogleTest is used for unit tests; pybind11 connects selected C++ components to Python; PyQt6 and pyqtgraph provide the desktop GUI; matplotlib is used by plotting utilities.

3.1 Technology stack

Core language: C++23 for order book, order entry, market-data publishing/receiving, recovery protocols, sequence tracking, hashing and metrics.

Build system: CMake (minimum version documented as 3.24). Compiler support documented for modern Clang and GCC toolchains.

Networking: TCP sockets for order entry and recovery; UDP multicast for market-data feeds; packed binary message formats and framed TCP protocol messages.

Concurrency and buffering: background publisher thread, ring buffers, bounded queues, mutexes, condition variables, recovery worker pools and per-subscriber scheduling queues.

Integrity and testing: SHA-256 snapshot/book digests, GoogleTest unit tests, Python end-to-end test runners and repeatable seeded fault injection.

Python tooling: Python 3.11 or later, pybind11, PyQt6, pyqtgraph and matplotlib for bindings, GUI, experiment orchestration and result visualisation.

Data output: CSV and structured JSON/stdout records for subscriber results, latency samples, recovery requests and experiment aggregation.

3.2 Repository structure and module responsibilities

lib/OrderBook: Security, Order, OrderBookEntry and OrderBook abstractions. Implements price-level organization, matching, market orders, cancellation, amendment and book statistics.

lib/OrderEntry: binary order messages, TCP client/server, connection handshake and bounded order queue.

lib/MDFeed: market-data message definitions, publisher, multicast publisher thread, multicast receiver, ring buffer and reliability subsystem.

lib/MDFeed/src/reliability: BookBuilder, DualFeed, FaultInjector, Metrics, RecoveryHistory, RecoveryProtocol, RecoveryScheduler, RecoveryTcp, RecoveryWire, SequenceTracker, Sha256 and Snapshot implementations.

src/exchange: Exchange lifecycle, symbol manager, command-line entry point, simulation and client-order processing.

src/order_client and src/workload_client: interactive and automated order-entry clients. src/reliable_subscriber and src/recovery_client: networked subscriber and recovery client applications.

src/reliability_demo: deterministic in-process reliability harness. src/reliability_runner: live experiment runner, preset experiments and aggregation. src/simulation_python: GUI and controllers.

tests: unit tests, live reliability end-to-end tests and GUI smoke tests. benchmarks: order-book benchmark. docs: design, protocol, metrics, implementation checklist, live-testing and viva guides.

3.3 Order entry and matching implementation

The order-entry server accepts multiple TCP clients. TCP is a byte stream, so the server cannot assume that one receive operation equals one complete order. It reconstructs complete frames, checks their declared sizes, performs the connection handshake and places complete messages into a bounded queue. The exchange consumes messages from the queue and dispatches new, cancel and compatible modify requests.

The OrderBook organizes bids from highest to lowest and asks from lowest to highest. An incoming limit order is compared with the best opposite-side price. If the prices cross, the matching engine executes quantity against available resting orders until the incoming quantity is exhausted or the limit price no longer crosses. A remaining limit quantity can rest at its price level. Cancellation removes an order by identifier. Amendment is implemented as removal followed by re-addition, which means the amended order loses its old queue position.

The repository documents an important limitation: the execution-report price is computed after matching from the current best opposite-side price and may differ from the actual resting execution price. Therefore the project demonstrates the order workflow but should not claim a fully exchange-grade execution-report model. The standalone order_server executable is also only a receive/print example; the actual exchange path is the Exchange application and OrderEntryServer.

3.4 Market-data publishing and transport

The MDAdapter translates order-book notifications into market-data publisher calls. The MarketDataPublisher assigns a global sequence number to each published message and places it in the publisher ring buffer. The MulticastPublisherThread drains the buffer asynchronously and sends datagrams using UDP sockets. Message definitions include headers, price-level update/delete, trade, snapshot-related and book-clear message types.

In the live exchange path, dual-feed mode sends the same existing binary datagram stream to two multicast endpoints. This should be distinguished from the separate in-process DualFeedPublisher abstraction used by the deterministic demo. The live subscriber joins configured multicast groups and validates message sizes before processing events. The base MulticastReceiver can count sequence discontinuities, but the reliable subscriber adds the full duplicate/gap/recovery logic.

3.5 Reliability and recovery implementation

SequenceTracker classifies events against the next expected sequence. DualFeedSubscriber performs first-copy-wins suppression, buffers later events during a gap, calls a gap callback, accepts recovered events and drains the pending map once continuity is restored. The implementation includes buffering for out-of-order recovery responses, which is important when multiple recovery workers complete requested slices in a different order.

RecoveryHistory stores a bounded deque of published events. RecoveryServer validates requests and returns status values including OK, TOO_OLD, BAD_REQUEST and ERROR. The in-process RecoveryClient calls the server object directly; the live RecoveryTcpClient and RecoveryTcpServer use a separate framed TCP connection. Large replay ranges can be returned in chunks and terminated with a completion frame.

Snapshots can be periodic and can also be created on demand when no suitable retained checkpoint exists. SnapshotManager controls checkpoint interval and retention. SnapshotStore

retains a bounded set. SnapshotHasher hashes canonical checkpoint metadata and price levels together with the previous checkpoint hash. The subscriber verifies the checkpoint before applying it. The hash chain detects changes to retained checkpoint content and links, but it is not a substitute for authenticated transport, durable storage, key management or external audit logs.

The live path also supports retrying transient recovery failures with exponential backoff and configurable recovery concurrency. Final reconciliation uses STATUS to establish the exchange's latest sequence, then verifies sequence accounting and the local book digest. This prevents an apparently quiet subscriber from declaring success while an unseen tail gap remains.

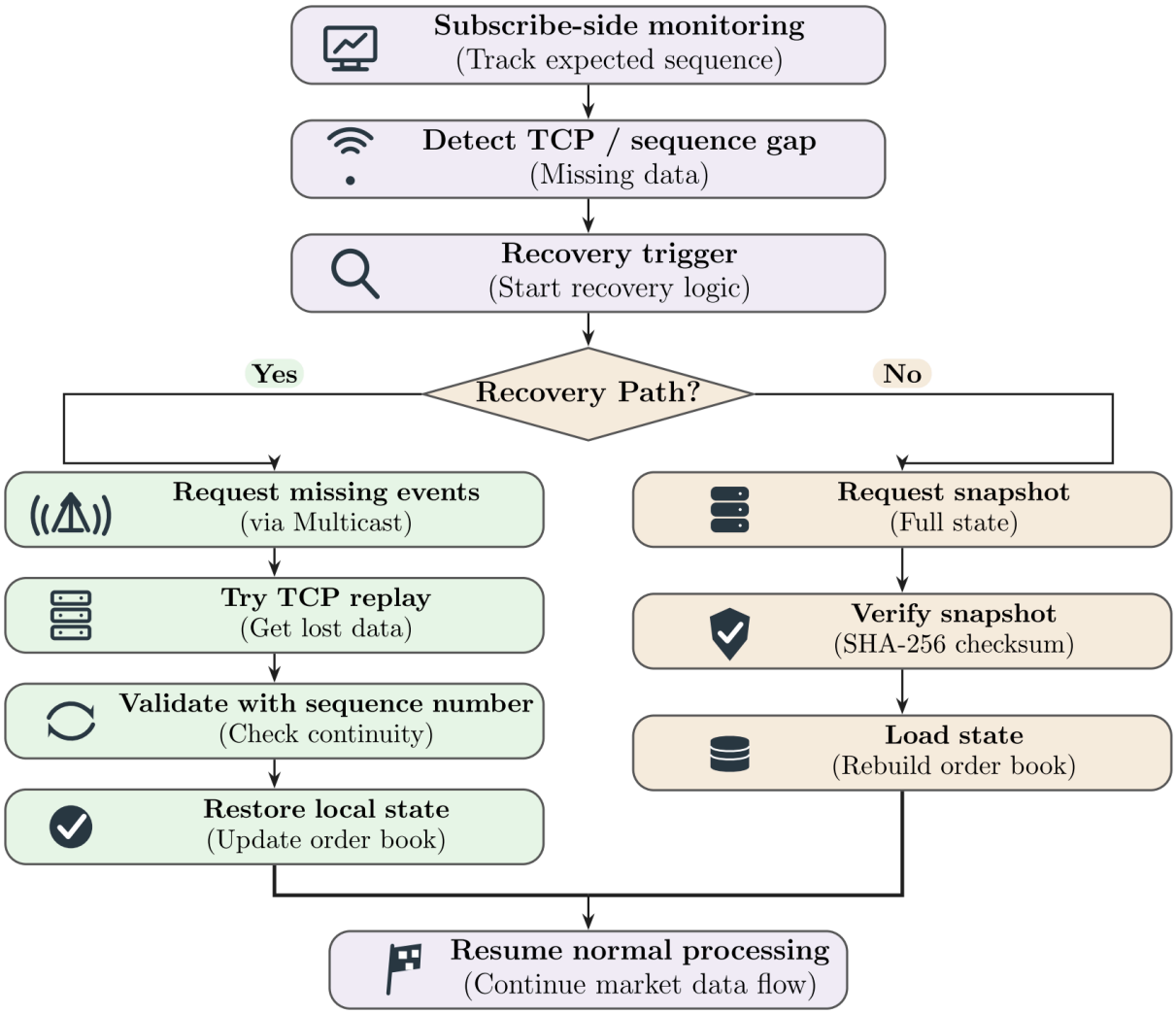


Fig 7. Reliability and recovery

3.6 Metrics and dashboard

The system records delivered events by source, gaps, duplicates, recovery ranges, snapshots, rejected snapshots, TOO_OLD responses, latency and book-match results. Latency is measured

per missing event from the instant a gap is detected until that event is delivered in order or covered by a verified snapshot. The documented classes are other_feed, tcp_recovery and snapshot; TCP request/response round-trip time is reported separately.

Percentiles include p50, p95, p99, p99.9 and maximum. The metrics documentation flags samples below 30 as insufficient for stable percentile interpretation and samples below 1000 as insufficient for p99.9. Dual-feed redundancy effectiveness is the fraction of Feed-A-missed sequences that arrive through Feed B. Recovery fairness uses per-subscriber queue-wait percentiles and a saturated-window Jain fairness index, which focuses on intervals when multiple subscriber queues are active.

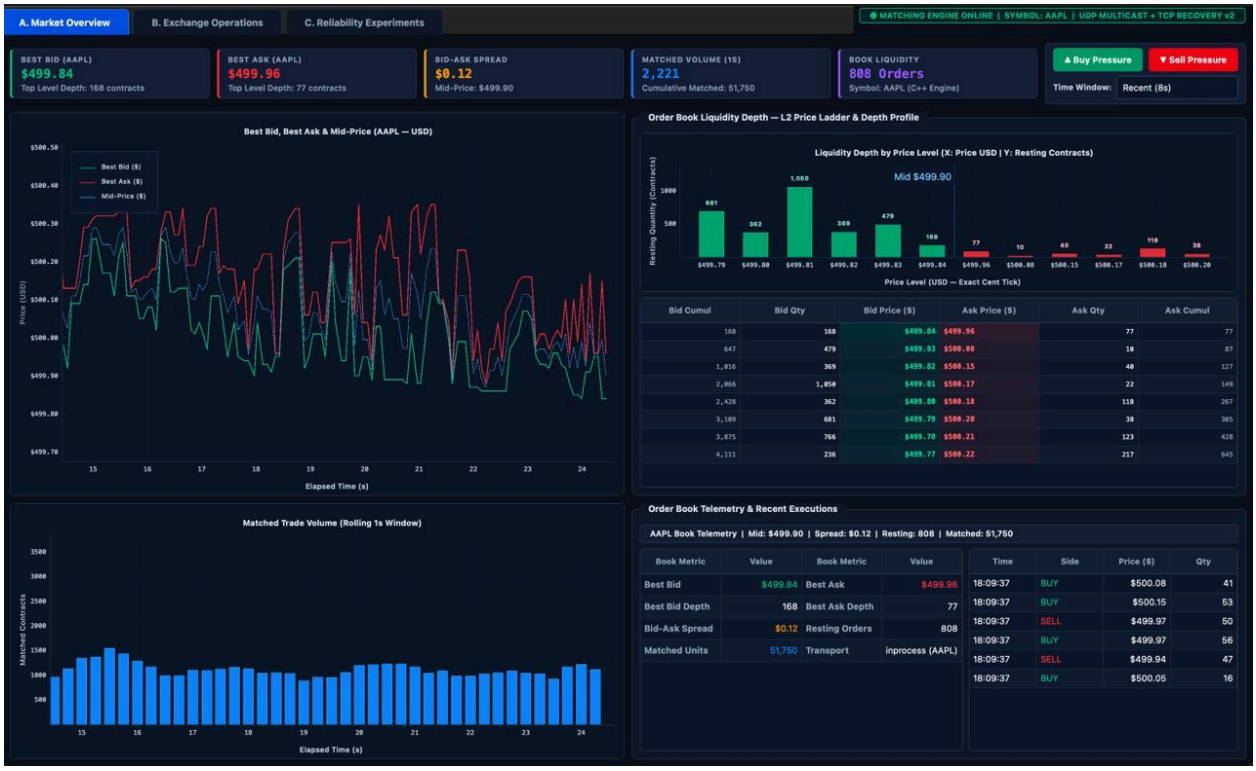
The GUI contains market visualisation, exchange control, manual TCP order testing, LAN setup, saved-results exploration and a reliability experiment dashboard. The reliability dashboard presents six data-driven charts and structured tables. The GUI's market visualisation calls the C++ order book through pybind11 with a NullMarketDataPublisher; therefore GUI simulator actions do not themselves publish UDP market data. Live exchange control is a separate process-based workflow.

3.7 Screenshots and visual evidence

The two architecture diagrams supplied by the project group are included in this report as Figures 1 and 2. No live application screenshots were supplied with the source ZIP or this request. To avoid presenting fabricated evidence, the report reserves space for actual screenshots of (

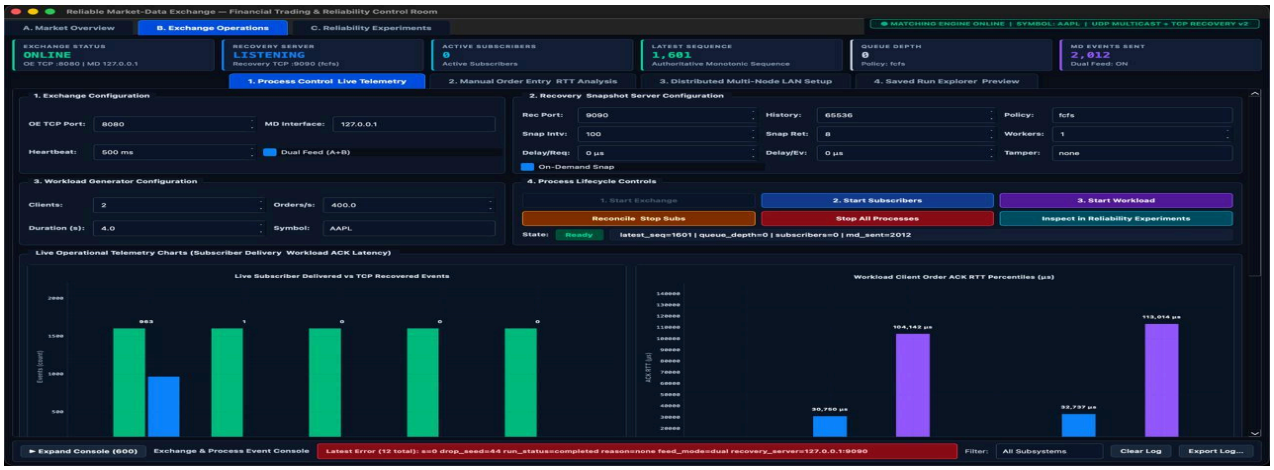
a) the Market Visualisation tab : Displays real-time best bid and ask prices, mid-price and bid–ask spread, along with price movements and matched trade volume.

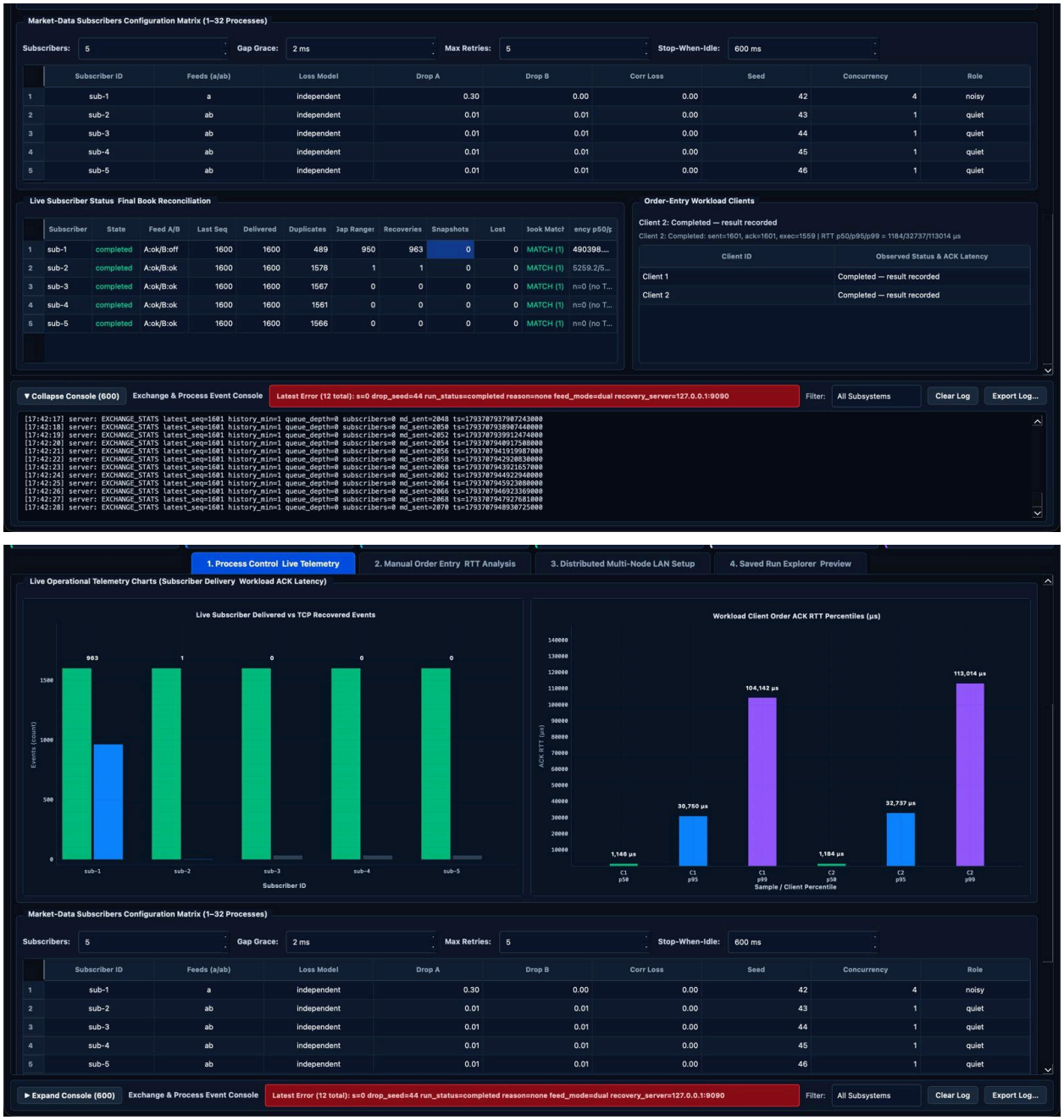
It also provides order-book depth, liquidity distribution, resting orders and recent trade executions, giving a consolidated view of the exchange's live market activity.



(b) Exchange Operations and Live Process Monitoring: Shows the exchange control interface, including server status, subscriber connections, order-entry configuration, recovery settings and workload controls.

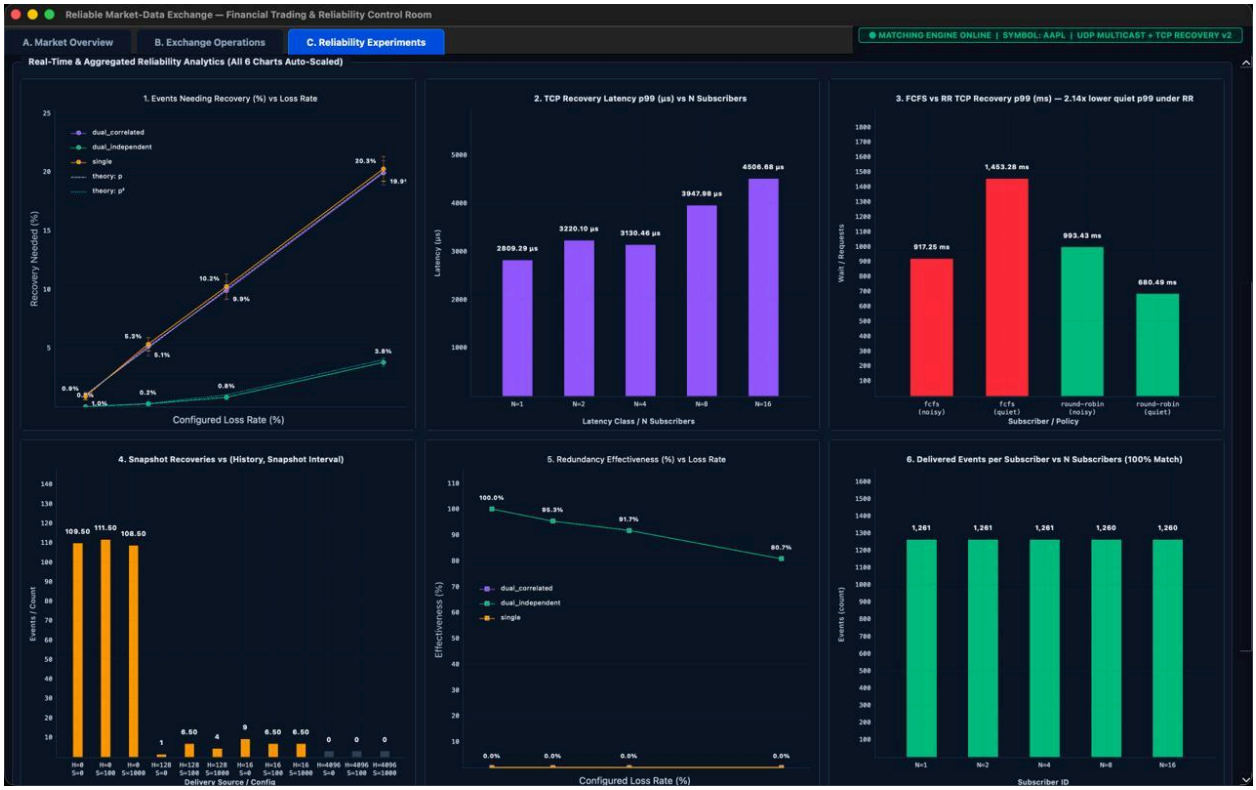
It also presents live delivery and latency graphs, subscriber configuration, final book reconciliation results, order acknowledgements and system logs, demonstrating the exchange's operational monitoring and control capabilities.





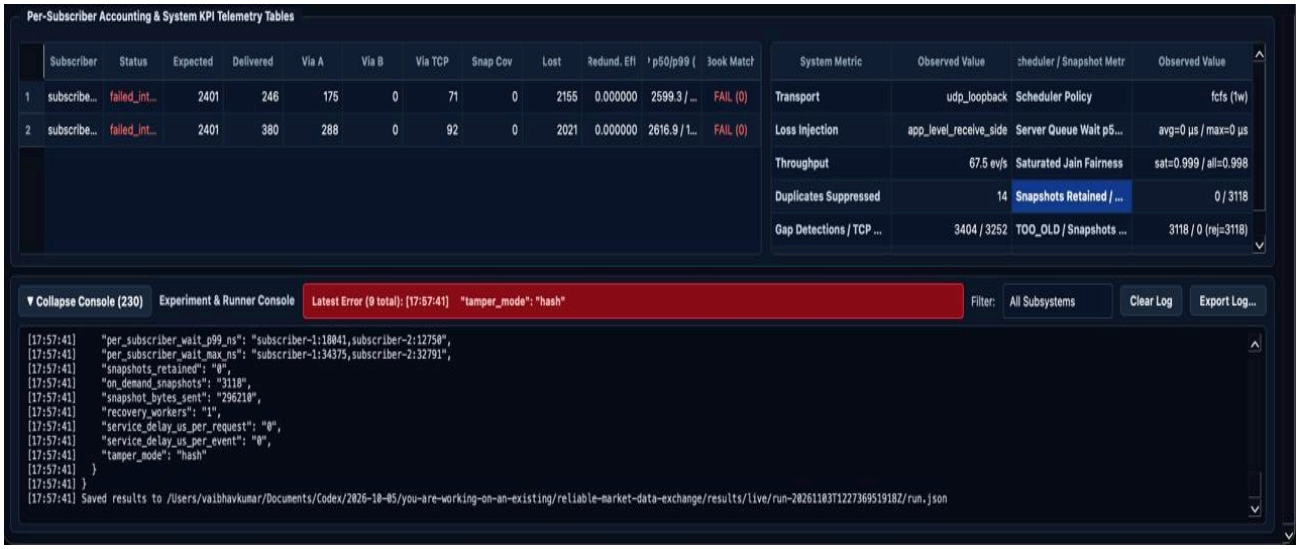
(c) Reliability Experiments and Performance Analytics: Presents six live graphs evaluating the exchange's reliability under different packet-loss conditions and subscriber loads.

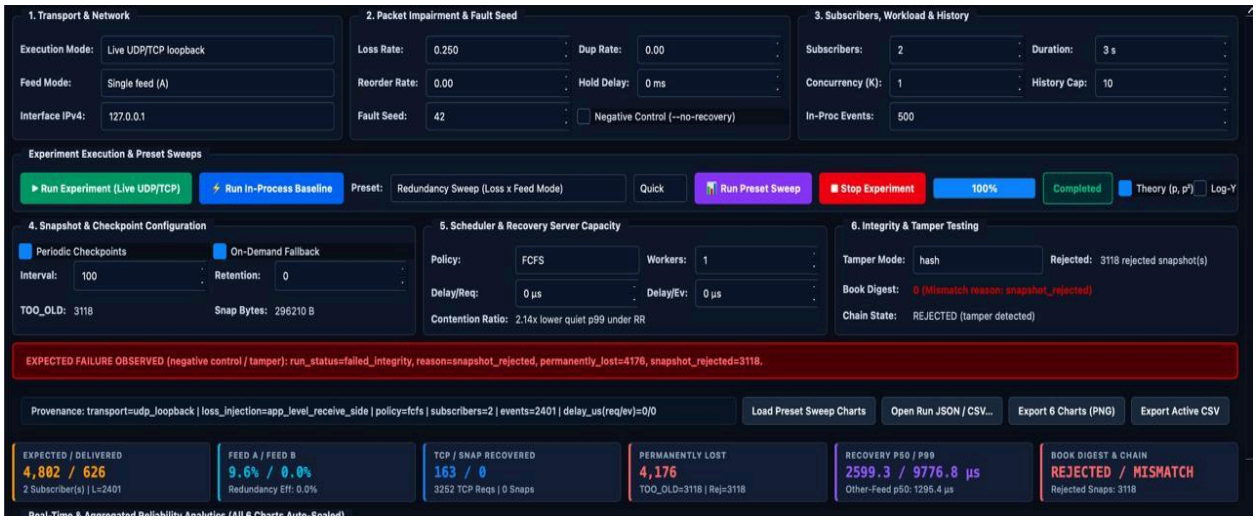
The graphs compare recovery requirements, TCP recovery latency, FCFS and Round-Robin scheduling, snapshot recovery, dual-feed redundancy effectiveness and successful event delivery across subscribers.



(d) Per-Subscriber Results and Integrity Testing: The upper section presents subscriber-wise delivery, recovery, packet-loss and book-matching statistics, along with system and scheduler metrics.

The lower section shows the experiment configuration and tamper-testing results, including snapshot rejection when hash verification detects an integrity mismatch.





4. Testing and Evaluation

Testing is performed at several levels. Unit tests cover isolated components such as order-book behaviour, sequence tracking, dual-feed delivery, snapshot hashing, recovery scheduling and impairment logic. Live end-to-end tests start separate exchange, workload-client and reliable-subscriber processes and check the actual TCP/UDP workflow. GUI smoke testing checks that the visual application can start and render in an offscreen environment. Preset experiments explore loss, redundancy, history size, snapshot fallback, scheduler contention and scalability.

4.1 Reported verification results

The repository's viva guide reports a C++23 build using AppleClang 17 and Homebrew GCC 16, with 51 of 51 GoogleTest cases passing. It reports 30 consecutive lossy runs with eight subscribers passing, and a 32-subscriber scalability run in which all 32 completed with no permanently lost events and matching book digests. It also reports a GUI smoke test passing.

For independent 10% feed loss with seed 42, the guide reports that Feed B repaired 91 of 101 potential Feed-A gaps (90.1%) immediately, reducing TCP recovery ranges from 88 to 8. The reported p50 latency was approximately 0.15 ms for repair from the other feed compared with approximately 3.15 ms for TCP recovery.

For a scheduler contention experiment, the repository reports quiet-subscriber p99 queue wait falling from 15.46 ms under FCFS to 5.07 ms under Round-Robin, a 3.05× improvement. The saturated-window Jain fairness index is reported as increasing from 0.625 to 0.946. For

on-demand snapshot fallback with a short history and 30% single-feed loss, it reports 20 snapshot recoveries, zero failures and a matching book. A tampered snapshot is reported as rejected by the subscriber.

These values are transcribed from the repository's documentation and verification ledger. They are not independently reproduced during preparation of this report. Results depend on the test host, configuration, workload, random seed, sample count and implementation version. They should be presented as project test results with their settings, not as universal performance guarantees.

Experiment / verification	Repository-reported result
GoogleTest unit suite	51/51 tests passed (reported 0.35 s)
30-run lossy flake gate	30/30 eight-subscriber lossy runs passed; zero reported failures
32-subscriber scalability	32/32 subscribers completed; permanently_lost=0; book_matches_exchange=1
Dual-feed redundancy (10% independent loss, seed 42)	91/101 Feed-A potential gaps avoided (90.1%); TCP recovery ranges reduced 88 to 8
Repair latency (same experiment)	Other-feed p50 about 0.15 ms; TCP recovery p50 about 3.15 ms
Scheduler contention	Quiet subscriber p99 queue wait: 15.46 ms FCFS vs 5.07 ms Round-Robin; 3.05× improvement
Saturated-window fairness	Jain index: 0.625 FCFS vs 0.946 Round-Robin
On-demand snapshot fallback	20 snapshot recoveries; zero failures; book_matches_exchange=1
Snapshot tamper test	Mutated snapshot rejected; integrity failure observed

4.2 Comparison with existing approaches

The comparison below is architectural.

Approach	Loss handling	Ordering / duplicates	Recovery when history expires	State verification	Main trade-off
Single UDP multicast feed	No inherent repair; loss can leave gaps	Application must add sequence logic	Usually absent or external	Not inherent	Low distribution overhead, weaker delivery assurance

UDP + TCP retransmission	Repairs detected gaps while retained	Sequence tracking and replay needed	Fails if requested data is no longer retained	Depends on implementation	Reliable repair, but recovery RTT and server load
Dual UDP feeds only	Second feed can mask independent loss	Subscriber must deduplicate and reorder	No solution if both feeds miss data	Not inherent	Reduces gaps but does not solve correlated loss or long outages
Proposed Group 11 system	Second feed, bounded TCP replay, snapshot fallback	First-copy-wins, gap buffering and ordered drain	Verified snapshot fallback	SHA-256 checkpoint chain and final book digest comparison	More components and configuration; better observable recovery path

4.3 Limitations and scope boundaries

The system is a centralised prototype, not a distributed matching engine. It does not implement consensus or replicated authoritative order books. UDP multicast and TCP recovery have been implemented in a real-process path, but deployment across production networks, multicast routing domains and real exchange infrastructure requires further validation.

The hash chain provides tamper evidence for checkpoint data but does not provide durable persistence, publisher authentication, encryption, key management or protection against an attacker who can rewrite the entire chain and trusted root. The report does not claim those properties.

The project contains both in-process and live paths; their results must not be mixed. In particular, the deterministic demo's callback-based feeds and empty-book snapshots are not the same as live UDP/TCP state reconstruction. The GUI's local order-book simulator is also not itself a live multicast publisher.

The order-entry execution report is approximate as documented. The public interactive client supports new and cancel orders; modify dispatch exists for compatible clients but is not exposed in the command-line UI. The system is not certified for production trading, high availability or regulatory compliance.

5. Conclusion and Future Work

The project demonstrates a complete educational workflow from client order submission to exchange matching and downstream market-data reconstruction. Its main contribution is the

integration of dual UDP multicast feeds, sequence-based gap detection, bounded TCP replay, snapshot fallback, SHA-256 integrity checks, recovery fairness scheduling and observable experiments around a central authoritative exchange.

The system illustrates why reliable market-data delivery requires more than sending packets. A subscriber must know which sequence it expects, detect discontinuities, avoid applying events out of order, recover missing information, verify the reconstructed state and reconcile the final tail of the stream. Redundant feeds reduce recovery demand, bounded history handles recent gaps, and snapshots provide a way to recover after history has expired.

Future work includes durable snapshot and event-history storage; authenticated and encrypted recovery transport; stronger publisher identity and key management; exact per-fill execution reports; richer order types and validation; more extensive cross-platform LAN testing; multicast network observability; load tests at larger event rates and subscriber counts; crash/restart recovery tests; and independent comparison against established open-source exchange/feed-handler implementations.

Bibliography

[1] J. Postel, "User Datagram Protocol," RFC 768, Internet Engineering Task Force (IETF), August 1980. Read RFC 768.

RFC Editor

[2] W. Eddy, "Transmission Control Protocol (TCP)," RFC 9293, Internet Engineering Task Force (IETF), August 2022. Read RFC 9293.

rfc-editor.org

[3]Internet Engineering Task Force, "Host Extensions for IP Multicasting," RFC 1112, August 1989. Read RFC 1112.

[4] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," FIPS PUB 180-4, August 2015. Read FIPS 180-4.

[5] ISO/IEC, "Programming Languages — C++," ISO/IEC 14882:2024. Use this if your project uses the C++23 standard; otherwise cite the standard version actually used.

[6] Google, "GoogleTest User Guide." GoogleTest documentation.

[7] Riverbank Computing, "PyQt6 Documentation." PyQt6 documentation.

[8] PyQtGraph Development Team, "PyQtGraph Documentation." PyQtGraph documentation.

[9] Python Software Foundation, "Python 3 Documentation." Python documentation.

[10] The OpenSSL Project, "OpenSSL Documentation." OpenSSL documentation. Include this only if your implementation actually uses OpenSSL.